Design-Weighted LCA: Data and Model Selection

Institutional trust in Mexico: LAPOP AmericasBarometer 2023

Kevin

2026-07-20

Purpose

This script is the heavy half of the pipeline: it presents the raw data and the dictionary, then searches over K with the design-weighted pseudo-likelihood, and, once the analyst hard-codes `cfg$K_force`, produces the model-fit diagnostics (item discrimination, bivariate residuals) and saves every object the segments script needs to `{cfg$out_dir}`. Nothing here is interpretive: interpretation, labeling, prediction, and the handoff live in `survey_lca_segments.qmd`, which renders in seconds because this script did the fitting.

The data

The stacked bars show the items as read, with missingness explicit: the grey mass is exactly what the complete-case rule removes. The dictionary pairs each analysis name with its source code, the question text, and the response labels as text, exactly what the segment labeler will later read; it is split across pages so it never overflows.

```
print(plot_item_stack(survey_dat_original, items,
                      "Raw items with missingness", show_missing = TRUE))
```

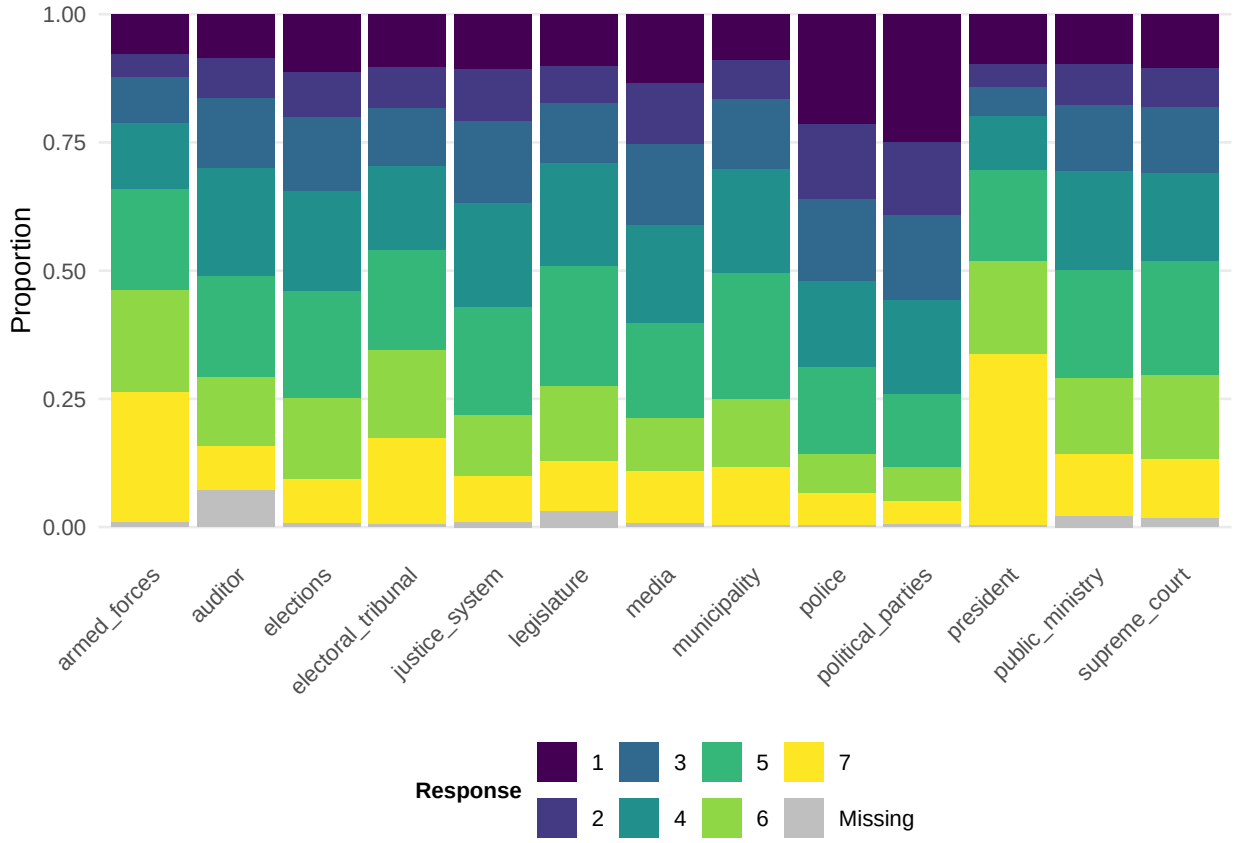

Figure 1: All respondents, all items, missingness shown explicitly (grey). The complete-case analysis sample removes rows contributing to the grey mass.

```
cat(nrow(survey_dat), "of", nrow(survey_dat_original),
    "cases are complete on items and demographics and enter the analysis.\n")
```

```
1430 of 1622 cases are complete on items and demographics and enter the analysis.
```

```
# Two tables so the dictionary fits PDF pages regardless of item count; the
# split is computed from length(items), nothing hard-coded. format = "pipe"
# lets Pandoc build a wrapping table (LaTeX tabular does not wrap long
# question text), and cat() guarantees emission under results: asis.
halves <- split(seq_along(items), ceiling(2 * seq_along(items) / length(items)))
walk(seq_along(halves), function(b) {
  cat(knitr::kable(dictionary[halves[[b]], ], format = "pipe",
                   col.names = c(
                     "Analysis name", "Source variable", "Survey question",
                                 "Response labels"),
                   caption = paste0("Item dictionary (", b, " of ",
                                    length(halves),
                                    "): the renamed analysis variable, its source variable ",
```

```
      sep = "\n")
  cat("\n\n")
  if (b < length(halves)) cat("\\newpage\n\n")
})
```

Table 1: Item dictionary (1 of 2): the renamed analysis variable, its source variable in the .sav, the question wording, and the response labels the model and the segment labeler read.

Analysis name	Source variable	Survey question	Response labels
justice_system	b10a	Confianza en el sistema de justicia	Nada | 2 | 3 | 4 | 5 | 6 | Mucho
electoral_tribunal	b11	Confianza en el Tribunal Supremo Electoral	Nada | 2 | 3 | 4 | 5 | 6 | Mucho
armed_forces	b12	Confianza en las fuerzas armadas	Nada | 2 | 3 | 4 | 5 | 6 | Mucho
legislature	b13	Confianza en la legislatura	Nada | 2 | 3 | 4 | 5 | 6 | Mucho
public_ministry	b15	Confianza en el Ministerio Público	Nada | 2 | 3 | 4 | 5 | 6 | Mucho
police	b18	Confianza en la policía nacional	Nada | 2 | 3 | 4 | 5 | 6 | Mucho

Table 2: Item dictionary (2 of 2): the renamed analysis variable, its source variable in the .sav, the question wording, and the response labels the model and the segment labeler read.

Analysis name	Source variable	Survey question	Response labels
auditor	b19	Confianza en la Auditoría Superior de la Federación	Nada | 2 | 3 | 4 | 5 | 6 | Mucho
political_parties	b21	Confianza en los partidos políticos	Nada | 2 | 3 | 4 | 5 | 6 | Mucho
president	b21a	Confianza en el presidente	Nada | 2 | 3 | 4 | 5 | 6 | Mucho
supreme_court	b31	Confianza en el tribunal supremo	Nada | 2 | 3 | 4 | 5 | 6 | Mucho
municipality	b32	Confianza en su municipalidad	Nada | 2 | 3 | 4 | 5 | 6 | Mucho
media	b37	Confianza en los medios de comunicación	Nada | 2 | 3 | 4 | 5 | 6 | Mucho
elections	b47a	Confianza en las elecciones	Nada | 2 | 3 | 4 | 5 | 6 | Mucho

Preparing the items

`prepare_items()` (source file) verifies the configured columns, coerces design variables to base types, recodes each item to consecutive integers over its substantive categories, and fixes the sum-to-n scale for the information criteria (rationale documented at its definition).

```
prep_fit     <- prepare_items(cfg)
dat_prepared <- prep_fit$dat_prepared
cats         <- prep_fit$cats
w_vec        <- prep_fit$w_vec
scale_ic     <- prep_fit$scale_ic
knitr::kable(prep_fit$summary, align = "lrr",
             caption = "Per-item category counts and missing footprint.")
```

Table 3: Per-item category counts and missing footprint.

item	categories	n_missing
justice_system	7	0
electoral_tribunal	7	0
armed_forces	7	0
legislature	7	0
public_ministry	7	0
police	7	0
auditor	7	0
political_parties	7	0
president	7	0
supreme_court	7	0

item	categories	n_missing
municipality	7	0
media	7	0
elections	7	0

Searching over K

Every candidate K is fit with `cfg$n_starts` seeded random starts; the table reports the weighted BIC and AIC on the sum-to-n scale, entropy, and the per-K inclusion probabilities. **Nothing is auto-selected**: read this evidence, set `cfg$K_force` in the config script, and re-render for that model's diagnostics.

```
# df_k() and entropy_R2() are defined in survey_lca_source.R; w_vec and the
# sum-to-n scale_ic came from prepare_items() (rationale documented there).
enum <- furrr::future_map_dfr(cfg$K_range, function(K) {
  fit  <- fit_lca(dat_prepared, w_vec, cats, cfg$items, K, cfg$n_starts)
  ll_n <- fit$ll * scale_ic                        # log-likelihood on the sum-to-n scale
  tibble(K = K,
         logLik = ll_n,
         df = df_k(K, cats),
         AIC = -2 * ll_n + 2 * df_k(K, cats),
         BIC = -2 * ll_n + df_k(K, cats) * log(nrow(dat_prepared)),
         entropy = entropy_R2(fit$post, K),
         converged = fit$converged)
}, .options = furrr::furrr_options(seed = TRUE,
     packages = c("matrixStats", "clue", "purrr", "dplyr", "tidyr")))

# K is the ANALYST's decision (cfg$K_force), made by iterating over the
# enumeration and the per-K diagnostics; nothing is auto-selected.
K_star <- as.integer(cfg$K_force %||% NA)

enum |>
  mutate(across(c(logLik, AIC, BIC), ~ round(.x, 1)),
         entropy = round(entropy, 3)) |>
  knitr::kable(align = "r",
               caption = "Weighted fit statistics by number of segments. Nothing is selected he
```

Table 4: Weighted fit statistics by number of segments. Nothing is selected here: read BIC, AIC, and entropy together and set cfg$K_force.

K	logLik	df	AIC	BIC	entropy	converged
2	-31579.7	157	63473.3	64300.0	0.904	TRUE
3	-30176.2	236	60824.3	62067.0	0.913	TRUE
4	-29447.7	315	59525.5	61184.1	0.921	TRUE

K	logLik	df	AIC	BIC	entropy	converged
5	-29047.7	394	58883.4	60957.9	0.907	TRUE
6	-28751.8	473	58449.5	60940.1	0.895	TRUE
7	-28537.7	552	58179.3	61085.8	0.893	TRUE
8	-28390.2	631	58042.5	61365.0	0.890	TRUE
9	-28242.7	710	57905.4	61643.8	0.890	TRUE
10	-28123.2	789	57824.4	61978.8	0.894	TRUE
11	-28024.0	868	57784.0	62354.4	0.902	TRUE
12	-27927.8	947	57749.6	62736.0	0.899	TRUE

```
enum |>
  dplyr::select(K, BIC, entropy) |>
  pivot_longer(-K, names_to = "criterion", values_to = "value") |>
  mutate(criterion = recode(criterion, BIC = "Weighted BIC", entropy = "Relative entropy")) |>
  ggplot(aes(K, value)) +
  geom_line(linewidth = 0.6, color = viridisLite::viridis(1, begin = 0.4)) +
  geom_point(size = 2.4, color = viridisLite::viridis(1, begin = 0.4)) +
  facet_wrap(~ criterion, scales = "free_y") +
  scale_x_continuous(breaks = cfg$K_range) +
  labs(x = "Number of latent classes (K)", y = NULL) +
  theme_lca()
```

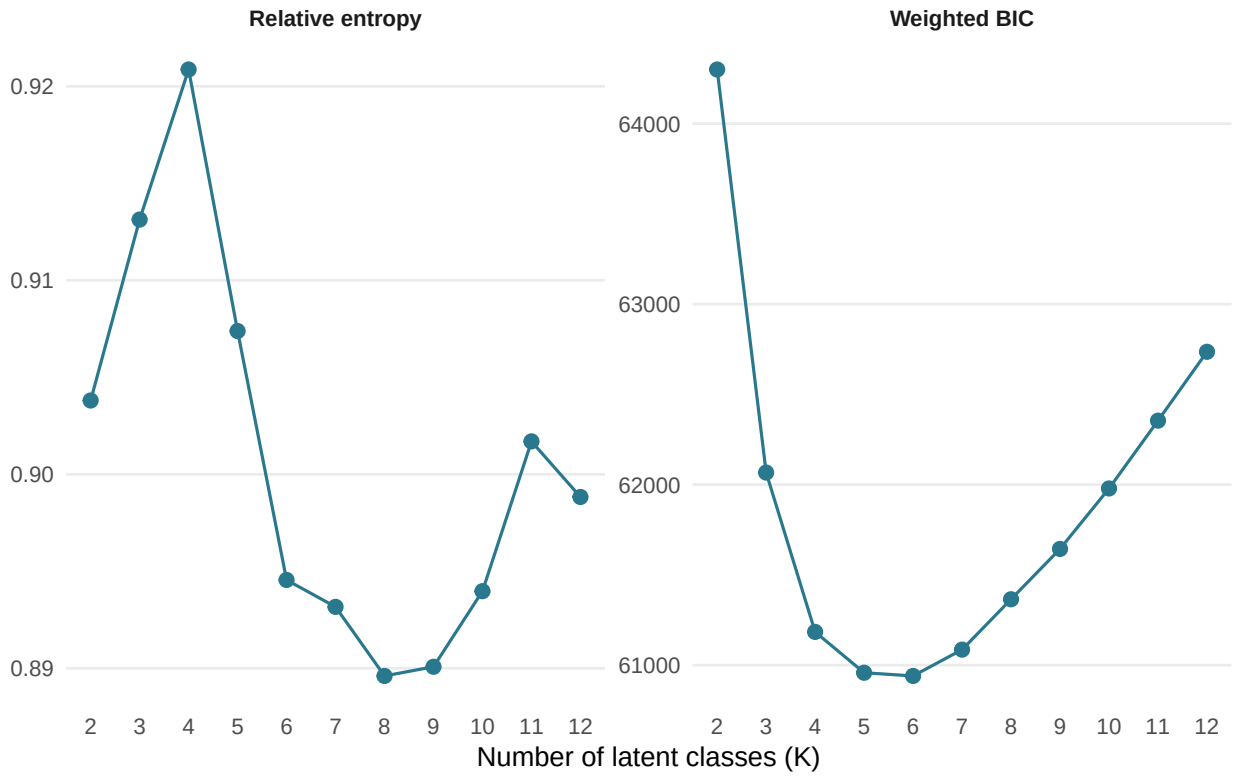

Figure 2: Enumeration diagnostics across K. Look for the BIC elbow and read entropy alongside it; neither decides alone.

```
if (!K_ready) cat("**`cfg$K_force` is `NULL`.** Review the enumeration evidence above, set a ca
```

The chosen model and its diagnostics

```
K_star <- as.integer(cfg$K_force)
# Fit at K_star on the complete cases, weighted and unweighted. The weighted fit sets
# the label order; the unweighted fit is aligned to it so "Class k" is consistent.
# Chosen-K fits run at terminal precision (poLCA's tolerance), so the
# unit-weight validation below compares optima rather than stopping rules.
fit_w <- fit_lca(dat_prepared, w_vec, cats, cfg$items, K_star, cfg$n_starts,
                 maxit = 5000L, tol = 1e-10)
fit_u <- align_to(fit_lca(dat_prepared, rep(1, nrow(dat_prepared)), cats, cfg$items,
                          K_star, cfg$n_starts, maxit = 5000L, tol = 1e-10), fit_w)

# Posteriors recomputed from the FINAL parameters, then written back so the same replicate
# design serves both the measurement-model CIs and the profiling regression.
inp_full  <- make_inputs(dat_prepared, cfg$items, cats)
post_w    <- posterior_of(fit_w$pi, fit_w$rho, inp_full$Y)
```

```r
colnames(post_w) <- paste0("post_class", seq_len(K_star))
modal_int <- max.col(post_w, ties.method = "first")
dat_prepared <- dat_prepared |>
  bind_cols(as_tibble(post_w)) |>
  mutate(modal_class   = factor(modal_int, levels = seq_len(K_star)),
         max_posterior = post_w[cbind(seq_len(n()), modal_int)])

# Report the fitted model: convergence, fit on the sum-to-n scale, classification
# sharpness, and the weighted prevalences that the segments script will re-present
# with design-based intervals.
cat(sprintf(paste0("Fitted K = %d on %d complete cases (%d random starts).\n",
                   "Converged: %s | weighted log-likelihood (sum-to-n scale): %.1f\n",
                   "Entropy R2: %.3f | mean maximum posterior: %.3f\n"),
            K_star, nrow(dat_prepared), cfg$n_starts,
            ifelse(isTRUE(fit_w$converged), "yes", "NO - inspect before trusting"),
            fit_w$ll * scale_ic, entropy_R2(post_w, K_star),
            mean(dat_prepared$max_posterior)))
```

```
Fitted K = 5 on 1430 complete cases (20 random starts).
Converged: yes | weighted log-likelihood (sum-to-n scale): -29047.7
Entropy R2: 0.907 | mean maximum posterior: 0.939
```

```r
knitr::kable(
  tibble(Segment = paste("Segment", seq_len(K_star)),
         `Weighted prevalence` = round(fit_w$pi, 3),
         `Unweighted prevalence` = round(fit_u$pi, 3),
         `Modal-assignment share` = round(as.numeric(prop.table(
           table(factor(modal_int, levels = seq_len(K_star))))), 3)),
  align = "lrrr",
  caption = "Segment prevalences at the chosen K. The weighted column is the population estimat
```

Table 5: Segment prevalences at the chosen K. The weighted column is the population estimate (design-based intervals appear in the segments script); the modal-assignment share is the sample share of respondents whose highest posterior is that segment, and differs from the model prevalence whenever classification is uncertain.

Segment	Weighted prevalence	Unweighted prevalence	Modal-assignment share
Segment 1	0.331	0.331	0.332
Segment 2	0.107	0.107	0.107
Segment 3	0.242	0.242	0.242
Segment 4	0.111	0.111	0.110
Segment 5	0.208	0.208	0.209

Which items carry the segmentation

```
# Discrimination of each item under the fitted weighted model: how differently the classes
# answer it, measured as the mean over class pairs of the total-variation (TV) distance
# between their class-conditional response distributions. 0 = all classes answer alike (a
# drop candidate); near 1 = strong separation. Loop-free: map over items, then over pairs.
# Computed here; shown in the figure below and in the screen table further down.
class_pairs <- utils::combn(K_star, 2, simplify = FALSE)   # list of c(a, b) class-index pairs

item_discrim <- tibble(
  item = cfg$items,
  discrimination = map_dbl(fit_w$rho, function(rho_j) {
    # rho_j is (categories x classes); rho_j[, k] is class k's response distribution.
    # TV distance for one pair of classes, then averaged over all pairs.
    mean(map_dbl(class_pairs, ~ 0.5 * sum(abs(rho_j[, .x[1]] - rho_j[, .x[2]]))))
  })
) |>
  arrange(desc(discrimination))

knitr::kable(item_discrim |> mutate(discrimination = round(discrimination, 3)),
             col.names = c("Item", "Discrimination"), align = "lr",
             caption = "Item discrimination: the mean over segment pairs of the total-variatio
```

Table 6: Item discrimination: the mean over segment pairs of the total-variation distance between their response distributions. 0 means every segment answers the item alike (a drop candidate); values near 1 mean the item separates segments sharply.

Item	Discrimination
justice_system	0.754
public_ministry	0.708
auditor	0.697
supreme_court	0.685
legislature	0.661
elections	0.654
political_parties	0.627
municipality	0.620
electoral_tribunal	0.618
police	0.588
media	0.490
armed_forces	0.484
president	0.452

```
item_discrim |>
  mutate(item = factor(item, levels = rev(item))) |>
  ggplot(aes(discrimination, item)) +
```

```
  geom_col(fill = viridisLite::viridis(1, begin = 0.45), width = 0.7) +
  geom_vline(xintercept = 0.1, linetype = "dashed", color = "grey55") +
  scale_x_continuous(limits = c(0, 1)) +
  labs(x = "Discrimination (mean pairwise total-variation distance)", y = NULL) +
  theme_lca()
```

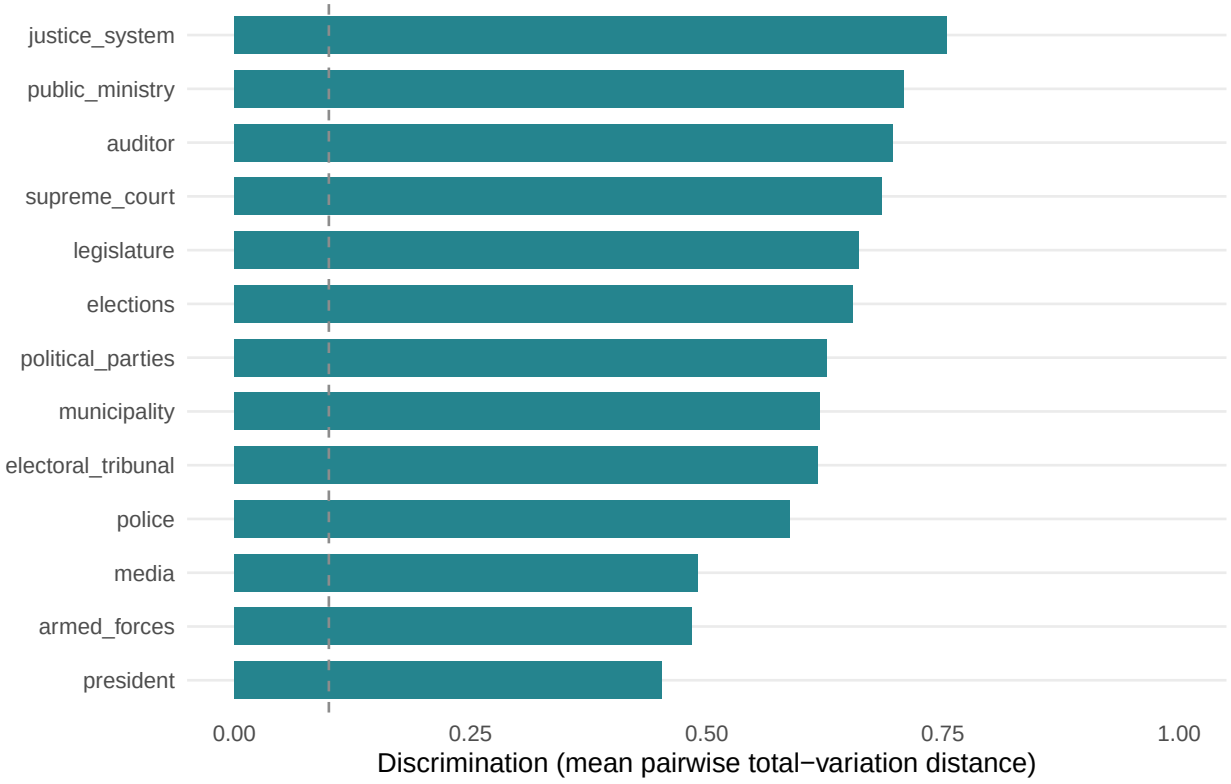

Figure 3: Item discrimination: how much each item separates the segments. Low bars identify candidates for removal in a sensitivity refit.

```
# Automated screen: does each item earn its class-DEPENDENT parameters by BIC? Compare, on
# the SAME respondents, the fitted model against a restricted model in which item j is made
# class-INDEPENDENT (its response distribution shared across classes, so it tells us nothing
# about the latent classes). Under local independence that restricted model factorises into
# an LCA on the OTHER items times item j's marginal, so its weighted pseudo-log-likelihood
# is ll(LCA without item j) + ll(item j's weighted marginal), and its BIC is directly
# comparable to the full model's. delta_bic = BIC(restricted) - BIC(full): NEGATIVE means
# BIC prefers dropping item j's class-dependence, so the item is flagged.
#
# This is the slowest chunk: it refits the LCA once per item. Lower cfg$n_starts here if you
# need speed, at some risk of a local optimum. It is reproducible because fit_lca() seeds
# each start from cfg$seed.
n_eff   <- nrow(dat_prepared)
# Put every log-likelihood on the same sum-to-n scale used for the criteria in
```

```
# Section 8 (scale_ic = n / sum(w_vec)), so the screen's BIC comparison is
# calibrated the same way as the K-selection BIC.
ll_full <- fit_w$ll * scale_ic
df_full <- df_k(K_star, cats)                    # df_k() was defined in Section 8

# Weighted log-likelihood of one item under a single (class-independent) distribution.
marginal_ll <- function(item) {
  y <- as.integer(dat_prepared[[item]])
  p <- as.numeric(xtabs(w_vec ~ y)); p <- p / sum(p)   # weighted response proportions, 1..C
  sum(w_vec * log(p[y]))
}

bic_screen <- tibble(item = cfg$items) |>
  mutate(delta_bic = furrr::future_map_dbl(cfg$items, function(j) {
    keep     <- setdiff(cfg$items, j)
    fit_wo   <- fit_lca(dat_prepared, w_vec, cats[keep], keep, K_star, cfg$n_starts)
    ll_restr <- (fit_wo$ll + marginal_ll(j)) * scale_ic            # sum-to-n scale, as above
    df_restr <- df_k(K_star, cats[keep]) + (cats[[j]] - 1)         # item j: one shared distril
    (-2 * ll_restr + df_restr * log(n_eff)) - (-2 * ll_full + df_full * log(n_eff))
  }, .options = furrr::furrr_options(seed = TRUE,
       packages = c("matrixStats", "clue", "purrr", "dplyr", "tidyr"))))

# Combine the effect size and the decision into one ranked, flagged table.
item_screen <- item_discrim |>
  left_join(bic_screen, by = "item") |>
  mutate(flag = if_else(delta_bic < 0, "drop candidate", "keep")) |>
  arrange(delta_bic)

item_screen |>
  mutate(discrimination = round(discrimination, 3), delta_bic = round(delta_bic, 1)) |>
  knitr::kable(align = "lrrl",
               col.names = c("Item", "Discrimination", "Delta BIC", "Screen"),
               caption = "Automated indicator screen, most-droppable first. Discrimination is t
```

Table 7: Automated indicator screen, most-droppable first. Discrimination is the mean pairwise total-variation distance (effect size). Delta BIC is BIC(item made class-independent) minus BIC(full model); a negative value means BIC prefers dropping the item's class-dependence, so it is flagged. The more negative the gap, the stronger the case (beyond about 6 is strong, beyond 10 very strong).

Item	Discrimination	Delta BIC	Screen
president	0.452	320.0	keep
armed_forces	0.484	424.0	keep
media	0.490	490.7	keep
police	0.588	742.1	keep
electoral_tribunal	0.618	827.5	keep

Item	Discrimination	Delta BIC	Screen
political_parties	0.627	863.6	keep
municipality	0.620	890.7	keep
legislature	0.661	987.7	keep
elections	0.654	988.1	keep
supreme_court	0.685	1116.4	keep
public_ministry	0.708	1161.9	keep
auditor	0.697	1197.1	keep
justice_system	0.754	1419.3	keep

Local independence: bivariate residuals

```
# bvr_pairs() is defined in survey_lca_source.R; fit$rho is positional in cfg$items
# order, so the items are passed in exactly that order.
bvr_tbl <- bvr_pairs(dat_prepared, w_vec, cfg$items, fit_w)

bvr_tbl |>
  mutate(bvr = round(bvr, 2)) |>
  slice_head(n = 10) |>
  knitr::kable(align = "llrr",
               col.names = c("Item A", "Item B", "Pair df", "BVR"),
               caption = "Largest bivariate residuals (BVR), a per-pair local-independence inde
```

Table 8: Largest bivariate residuals (BVR), a per-pair local-independence index: the design-weighted observed two-way table against the model-implied table, as Pearson X2 / (n-scaled) per degree of freedom. Descriptive ranking only; no chi-square test applies under the weighted pseudo-likelihood. Pairs standing well above the rest of the table mark associations the classes do not explain; absolute SRS cutoffs do not transfer under unequal weights.

Item A	Item B	Pair df	BVR
public_ministry	auditor	36	5.75
armed_forces	legislature	36	5.54
justice_system	supreme_court	36	3.21
police	political_parties	36	2.81
president	supreme_court	36	2.48
president	elections	36	2.40
electoral_tribunal	public_ministry	36	2.36
police	municipality	36	2.32
justice_system	elections	36	2.19
armed_forces	police	36	2.14

```r
# Column names come from bvr_pairs() (item_a, item_b, df, bvr); axis order is
# cfg$items, so this adapts to any item set with no hard coding.
bvr_tbl |>
  mutate(item_a = factor(item_a, levels = cfg$items),
         item_b = factor(item_b, levels = cfg$items)) |>
  ggplot(aes(item_a, item_b, fill = bvr)) +
  geom_tile() +
  geom_text(aes(label = sprintf("%.1f", bvr)), size = 2.0) +
  scale_fill_viridis_c(name = "BVR", direction = -1) +
  labs(x = NULL, y = NULL) +
  theme_lca() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

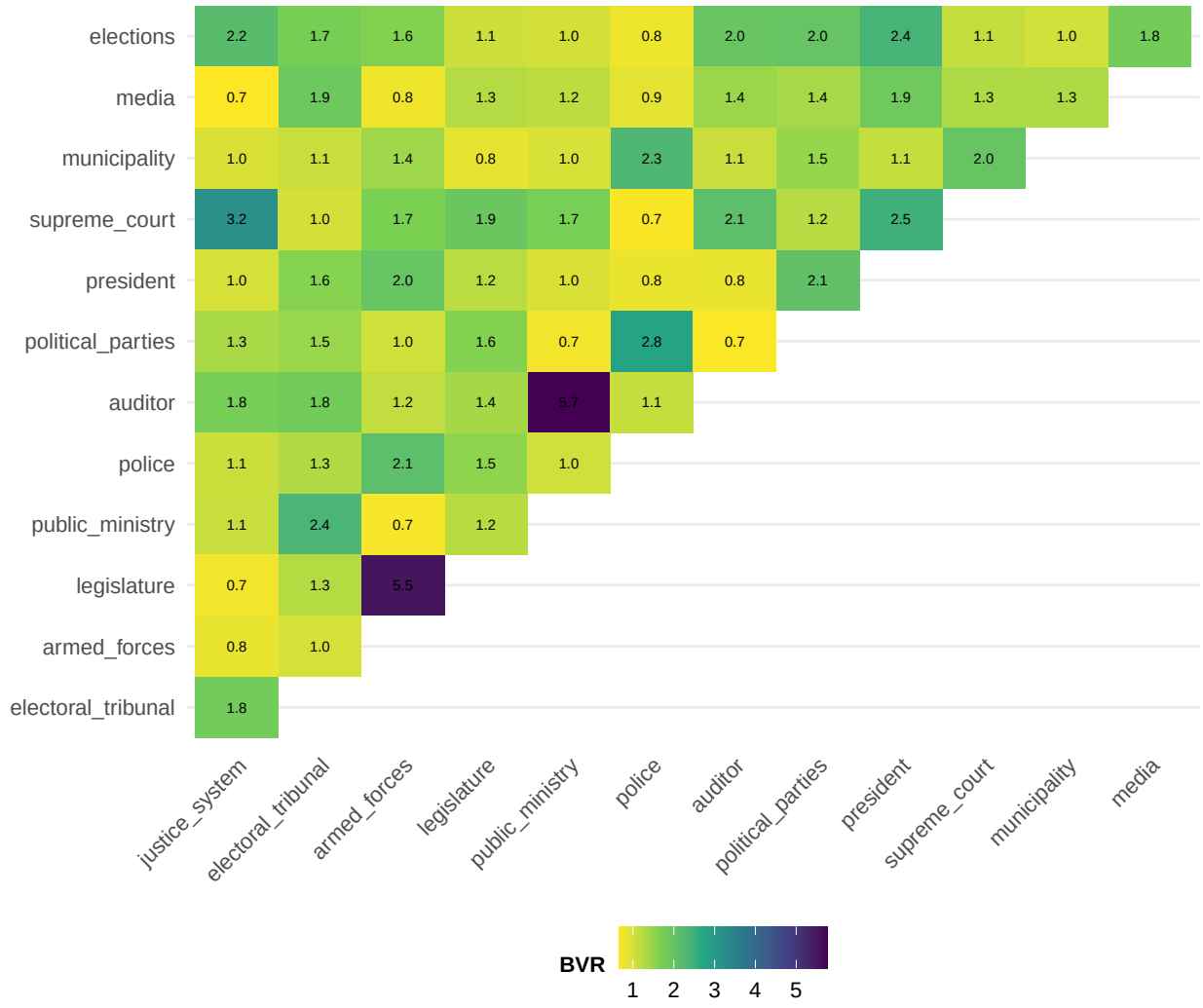

Figure 4: Bivariate residuals as a heatmap: darker cells are item pairs whose association the segments explain least well (a descriptive ranking, not a test). Persistent dark pairs argue for dropping/merging an item or trying K+1.

Design-based confidence intervals (computed here, presented in the segments script)

```
options(survey.lonely.psu = "adjust")

# Report PSU structure so the analyst sees the degrees of freedom they actually have.
# Note: dplyr::count() is namespace-qualified because matrixStats (loaded for
# rowLogSumExps) also exports a count(), and it loads after dplyr here.
psu_structure <- dat_prepared |>
  dplyr::distinct(.data[[cfg$strata]], .data[[cfg$psu]]) |>
  dplyr::count(.data[[cfg$strata]], name = "psus_in_stratum")
n_psu_total <- nrow(dplyr::distinct(dat_prepared, .data[[cfg$psu]]))
n_strata    <- n_distinct(dat_prepared[[cfg$strata]])
lonely_any  <- any(psu_structure$psus_in_stratum < 2)

# Declare the design and build stratified jackknife replicate weights.
des <- svydesign(ids = as.formula(paste0("~", cfg$psu)),
                 strata = as.formula(paste0("~", cfg$strata)),
                 weights = as.formula(paste0("~", cfg$weight)),
                 data = dat_prepared, nest = TRUE)
rep_des <- as.svrepdesign(des, type = "JKn")

# Parameter bookkeeping as a TIBBLE, in the exact order flatten_fit() produces.
# This carries item/category/class as real columns, so nothing is recovered from
# string parsing later and the survey-question names are preserved downstream.
param_meta <- bind_rows(
  tibble(kind = "pi", item = NA_character_, category = NA_integer_, class = seq_len(K_star)),
  imap_dfr(cats, function(Cj, item_name)
    expand_grid(class = seq_len(K_star), category = seq_len(Cj)) |>   # category varies fastest
      transmute(kind = "rho", item = item_name, category, class))
)
param_names <- param_meta |>
  mutate(nm = if_else(kind == "pi",
                      sprintf("pi[%d]", class),
                      sprintf("rho[%s,cat%d,class%d]", item, category, class))) |>
  pull(nm)
flatten_fit <- function(fit) set_names(c(fit$pi, unlist(map(fit$rho, as.vector))), param_names)

# The statistic survey::withReplicates evaluates on each replicate: refit the weighted EM
# on the replicate weights, warm-started at the full-sample fit and aligned to it.
theta_fun <- function(w_rep, data) {
  inp <- make_inputs(data, cfg$items, cats)
  fit <- em_run(inp$Y, inp$OH, cats, w_rep, K_star,
                init = fit_w[c("pi", "rho")], maxit = 400L)
  flatten_fit(align_to(fit, fit_w))
}
```

```r
rep_var <- withReplicates(rep_des, theta_fun)
est_vec <- flatten_fit(fit_w)
se_vec  <- sqrt(diag(vcov(rep_var)))

# Tidy table of every probability with its design-based 95% CI: item/category/class
# are columns, ready to filter, join, and plot.
# Intervals: logit-scale Wald via the delta method (se_logit = se / (p(1-p))),
# back-transformed so they respect (0, 1) without truncation, with the design
# t critical value (df = PSUs - strata) instead of 1.96. The same construction
# serves the domain estimates in the segments script, so one interval method
# runs through the whole pipeline. Boundary cells (estimate within eps of 0 or
# 1) have no meaningful logit SE; they fall back to Wald truncated to [0, 1]
# and are flagged in `boundary` so downstream tables can footnote them.
crit <- qt(0.975, df = survey::degf(rep_des))
eps  <- 1e-3
ci_tbl <- param_meta |>
  mutate(estimate = as.numeric(est_vec), se = as.numeric(se_vec),
         boundary = estimate < eps | estimate > 1 - eps,
         p_c = pmin(pmax(estimate, 1e-6), 1 - 1e-6),
         gse = se / (p_c * (1 - p_c)),
         lo = if_else(boundary, pmax(0, estimate - crit * se),
                      plogis(qlogis(p_c) - crit * gse)),
         hi = if_else(boundary, pmin(1, estimate + crit * se),
                      plogis(qlogis(p_c) + crit * gse))) |>
  dplyr::select(-p_c, -gse)

# What the design actually supports, then confirmation that every parameter got
# an interval.
cat(sprintf(paste0("Design: %d strata, %d PSUs -> %d jackknife replicates",
                   " (%d residual df).\n%s",
                   "Design-based SEs computed for %d parameters",
                   " (%d prevalences, %d conditional probabilities).\n"),
            n_strata, n_psu_total, ncol(rep_des$repweights), n_psu_total - n_strata,
            ifelse(lonely_any,
                   "Warning: at least one stratum contributes a single PSU; survey.lonely.psu =
                   ""),
            nrow(ci_tbl), K_star, nrow(ci_tbl) - K_star))
```

```
Design: 4 strata, 130 PSUs -> 130 jackknife replicates (126 residual df).
Design-based SEs computed for 460 parameters (5 prevalences, 455 conditional probabilities).
```

Unweighted benchmark (poLCA) for the validation table

```
# poLCA needs a data frame of items coded 1..C with no NA (it uses na.rm).
polca_df <- dat_prepared |> dplyr::select(all_of(cfg$items)) |> as.data.frame()
polca_f  <- as.formula(paste0("cbind(", paste(cfg$items, collapse = ", "), ") ~ 1"))

# --- substantive benchmark: poLCA at K_star, unweighted ---
set.seed(cfg$seed + 7L)
pl <- poLCA(polca_f, data = polca_df, nclass = K_star, nrep = 10,
            maxiter = 5000, na.rm = TRUE, verbose = FALSE)
pl_aligned <- align_to(list(pi = as.numeric(pl$P),
                            rho = map(pl$probs, t),
                            post = pl$posterior), fit_w)

# --- poLCA's own BIC-selected K across the candidate range ---
polca_bic <- furrr::future_map_dfr(cfg$K_range, function(K) {
  set.seed(cfg$seed + 10L + K)
  m <- poLCA(polca_f, data = polca_df, nclass = K, nrep = 5,
             maxiter = 5000, na.rm = TRUE, verbose = FALSE)
  tibble(K = K, poLCA_BIC = m$bic)
}, .options = furrr::furrr_options(seed = TRUE, packages = "poLCA"))
polca_K <- polca_bic |> slice_min(poLCA_BIC, n = 1) |> pull(K) |> first()

# --- validation: weighted EM with all weights = 1 should match poLCA ---
# fit_u (Section 5) is the unit-weight multi-start fit, already aligned to
# fit_w. Same likelihood, third candidate: EM warm-started at poLCA's own
# solution; whichever basin is higher wins. pl_aligned$rho is already
# (categories x classes) per item, em_run's format.
fit_from_pl <- em_run(inp_full$Y, inp_full$OH, cats,
                      rep(1, nrow(dat_prepared)), K_star,
                      init = list(pi = pl_aligned$pi, rho = pl_aligned$rho),
                      maxit = 5000L, tol = 1e-10)
fit_unit <- if (fit_from_pl$ll > fit_u$ll) align_to(fit_from_pl, fit_w) else fit_u
max_pi_gap  <- max(abs(fit_unit$pi - pl_aligned$pi))
max_rho_gap <- max(map2_dbl(fit_unit$rho, pl_aligned$rho, ~ max(abs(.x - .y))))

# The trust check: at unit weights our EM and poLCA maximize the SAME likelihood,
# so any gap beyond optimizer tolerance is an implementation problem, not a
# modeling result. The segments script reprints the prevalence verdict alongside
# the weighted estimates.
cat(sprintf(paste0("VALIDATION (unit weights, our EM vs poLCA):\n",
                   "  max |prevalence gap|            = %.5f\n",
                   "  max |conditional-probability gap| = %.5f\n  -> %s\n",
                   "poLCA's own BIC-minimizing K over %s: %d",
                   " (evidence, not a decision; K stays cfg$K_force = %d).\n"),
            max_pi_gap, max_rho_gap,
            ifelse(max(max_pi_gap, max_rho_gap) < 0.005,
                   "MATCH: implementations agree.",
```

```
                  "MISMATCH: investigate before trusting the weighted fit."),
          paste0(min(cfg$K_range), "-", max(cfg$K_range)), polca_K, K_star))
```

```
VALIDATION (unit weights, our EM vs poLCA):
  max |prevalence gap|            = 0.00000
  max |conditional-probability gap| = 0.00000
  -> MATCH: implementations agree.
poLCA's own BIC-minimizing K over 2-12: 6 (evidence, not a decision; K stays cfg$K_force = 5).
```

```
ll_ours <- fit_unit$ll; ll_polca <- pl$llik
cat(sprintf("log-likelihood  ours = %.4f | poLCA = %.4f | difference = %+.6f\n",
            ll_ours, ll_polca, ll_ours - ll_polca))
```

```
log-likelihood  ours = -29046.7917 | poLCA = -29046.7917 | difference = +0.000000
```

```
cat(sprintf("convergence     ours = %s | poLCA used %d iterations\n",
            ifelse(isTRUE(fit_unit$converged), "converged", "NOT converged"),
            pl$numiter))
```

```
convergence     ours = converged | poLCA used 92 iterations
```

```
# Positional indexing on BOTH lists: our EM's rho is unnamed while the poLCA
# side is named, so name-based lookup silently returns NULL. arrayInd() finds
# the largest gap without assuming which() returns a matrix.
gaps <- map_dfr(seq_along(cfg$items), function(j) {
  a <- as.matrix(fit_unit$rho[[j]]); b <- as.matrix(pl_aligned$rho[[j]])
  d <- abs(a - b); idx <- arrayInd(which.max(d), dim(d))
  tibble(item = cfg$items[j], category = idx[1], segment = idx[2],
         ours = a[idx[1], idx[2]], polca = b[idx[1], idx[2]], gap = max(d))
}) |> arrange(desc(gap))
print(head(gaps, 5))
```

```
# A tibble: 5 x 6
  item               category segment  ours polca          gap
  <chr>                 <int>   <int> <dbl> <dbl>        <dbl>
1 justice_system            1       2 0.652 0.652 0.0000000434
2 legislature               6       1 0.130 0.130 0.0000000395
3 supreme_court             4       3 0.231 0.231 0.0000000357
4 armed_forces              1       2 0.419 0.419 0.0000000330
5 electoral_tribunal        1       2 0.537 0.537 0.0000000310
```

```
cat("segment sizes:", paste(sprintf("%.3f", fit_unit$pi), collapse = ", "), "\n")
```

```
segment sizes: 0.336, 0.107, 0.239, 0.111, 0.207
```

```
# ONE validation verdict, frozen to disk; the segments script prints from this
# file instead of recomputing with a different criterion.
validation <- tibble(max_pi_gap, max_rho_gap,
                     ll_ours = fit_unit$ll, ll_polca = pl$llik,
                     polca_K = polca_K,
                     match = max(max_pi_gap, max_rho_gap) < 0.005)
```

Saving the objects for the segments script

Tables go to parquet (arrow), model objects (lists) to rds; `survey_dat` itself is NOT saved, both scripts rebuild it identically from `survey_data_config.R`, which keeps one source of truth for the data.

```
saveRDS(fit_w,       file.path(cfg$out_dir, "fit_weighted.rds"))
saveRDS(fit_unit,    file.path(cfg$out_dir, "fit_unweighted.rds"))
saveRDS(pl_aligned,  file.path(cfg$out_dir, "polca_aligned.rds"))
arrow::write_parquet(enum,    file.path(cfg$out_dir, "enumeration.parquet"))
arrow::write_parquet(ci_tbl,  file.path(cfg$out_dir, "ci_tbl.parquet"))
arrow::write_parquet(bvr_tbl, file.path(cfg$out_dir, "bvr_tbl.parquet"))
arrow::write_parquet(validation, file.path(cfg$out_dir, "validation.parquet"))
cat("Saved fit and tables for K =", K_star, "to", cfg$out_dir, "\n")
```

```
Saved fit and tables for K = 5 to ../output/mexico
```